

Conditionally Trivial Special Member Functions

Contents

1. [Motivation](#)
2. [Proposal](#)
3. [Design Considerations](#)
4. [References](#)

1. Motivation

There are many situations where we need to write a class template whose special member functions have the same underlying traits as the class template parameters. The most well-known cases of this are the new "sum" types in the C++ standard library: `std::optional<T>`, `std::variant<Ts...>`, and the proposed `std::expected<T, E>`. In each case, each class template specialization needs to be copyable if the underlying types are copyable, trivially copyable if the underlying types are trivially copyable, and trivially destructible if the underlying types are trivially destructible. These aren't just "nice-to-have" features either, they are important for several reasons:

- If a type is trivially destructible (among other criteria), it qualifies as a *literal type*, making it `constexpr`-friendly.
- If a type is trivially destructible and trivially copyable, the SysV ABI allows it to be passed via a register instead of a pointer.
- These features stack - so types containing these types would then be able to become literal and so forth.
- If a type advertises itself as satisfying some capability (e.g. copyable), but isn't, that leads to much more difficult compile errors to reason about than if the type was honest.

The problem is... it's actually quite difficult to implement this feature. Or rather, it's not so much that it's difficult, as that it's very verbose, tedious, repetitive, and error prone. What follows is a simplified implementation of `std::optional<T>` which is conditionally copyable, conditionally trivially copyable, and conditionally trivially destructible. It is, to the best of the author's knowledge, the simplest such solution:

```
template <typename T,
    bool trivial = /* ... */>
struct base_payload {
    struct empty { };
    union {
        empty _;
        T value;
    };
    bool engaged = false;

    base_payload() = default;

    base_payload(base_payload const& rhs)
        : engaged(rhs.engaged)
    {
        if (rhs.engaged) {
            new (&value) T(rhs.value);
        }
    }

    ~base_payload() {
        if (engaged) value.~T();
    }
};

template <typename T>
struct base_payload<T, true> {
    struct empty { };
```

```

union {
    empty _;
    T value;
};
bool engaged = false;

base_payload() = default;
base_payload(base_payload const& ) = default;
~base_payload() = default;
};

template <bool allow_copying>
struct copy_enabler {
    copy_enabler() = default;
    copy_enabler(copy_enabler const& ) = delete;
    ~copy_enabler() = default;
};

template <>
struct copy_enabler<true> {
    copy_enabler() = default;
    copy_enabler(copy_enabler const& ) = default;
    ~copy_enabler() = default;
};

template <typename T>
class optional
    : private copy_enabler</* copyable */>
    , private base_payload<T>
{
public:
    using base_payload<T>::base_payload;
    optional(optional const& ) = default;
    ~optional() = default;
};

```

That's 63 lines of code and all we've got is a default constructor, a copy constructor, and a destructor. But we do have two specializations of `base_payload` and two specializations of `copy_enabler`, and we have the storage of `optional` duplicated. This code does not really represent the intent of the programmer at all: it's simply the best available workaround. It's also difficult to follow, as some constructors will be in the `optional` derived type itself, but some will be inherited. In effect, we've split our class into five pieces, all of which need to be understood together - but they will potentially be hundreds of lines of code apart. This is all... less than great.

Thankfully, now we have Concepts.

2. Proposal

With Concepts, we can, for the first time, truly overload special member functions within the same class or class template specialization. That, in of itself, is a very powerful feature which solves the vast majority of the problem. All that's left is a little bit of wording detail.

The current criteria for trivially copyable reads, from [class]/6:

A trivially copyable class is a class:

- where each copy constructor, move constructor, copy assignment operator, and move assignment operator ([class.copy], [over.ass]) is either deleted or trivial,
- that has at least one non-deleted copy constructor, move constructor, copy assignment operator, or move assignment operator, and
- that has a trivial, non-deleted destructor.

A trivial class is a class that is trivially copyable and has one or more default constructors, all of which are either trivial or deleted and at least one of which is not deleted.

These rules don't take Concepts into account. It's an unnecessary restriction that *each* special member function be trivial or deleted if some of these can never be invoked and would never have its code generated. Hence, this paper proposes an update.

We call a special member function **usable** if it would be selected during overload resolution for its specific argument list. In other words, only the most constrained special member function with a specific signature is usable. For instance:

```

struct X {
    X(X const& ); // #1

    template <class... Args>
    X(Args&&... ); // #2
};

template <typename T>
struct Y {
    Y(Y const& ); // #3
    Y(Y const& ) requires std::is_pointer_v<T>; // #4
    Y(Y& ); // #5
};

```

#1 is usable (even if it were not user-specified). It doesn't matter attempting to construct an `X` from a non-const `X` would invoke #2. Exactly one of #3 or #4 is usable for all types: #4 for pointer types, #3 otherwise. #5 is always usable - the other copy constructors have different signatures.

With this term, this paper proposes new wording for two sections.

[class]/6:

A *trivially copyable class* is a class:

- where each [usable](#) copy constructor, move constructor, copy assignment operator, and move assignment operator ([class.copy], [over.ass]) is either deleted or trivial,
- that has at least one [usable](#) non-deleted copy constructor, move constructor, copy assignment operator, or move assignment operator, and
- that has a [usable](#) trivial, non-deleted destructor.

A *trivial class* is a class that is trivially copyable and has one or more [usable](#) default constructors, all of which are either trivial or deleted and at least one of which is not deleted.

and [basic.types]/10:

A type is a *literal type* if it is:

- [...]
- a possibly cv-qualified class type that has all of the following properties:
 - it has a [usable](#) trivial destructor
 - [...]

Additionally, [class.dtor]/12 would need to be updated to handle the degenerate situation where a class has no usable destructor.

These minor changes in the wording allow for a rewrite of the reduced `std::optional<T>` example that perfectly satisfies all of our criteria:

C++17	Proposed
<pre> template <typename T, bool trivial = /* ... */> struct base_payload { struct empty { }; union { empty _; T value; }; bool engaged = false; base_payload() = default; base_payload(base_payload const& rhs) : engaged(rhs.engaged) { if (rhs.engaged) { new (&value) T(rhs.value); } } } </pre>	<pre> template <typename T> class optional { struct empty { }; union { empty _; T value; }; bool engaged = false; public: constexpr optional() = default; optional(optional const&) requires /* trivially copyable */ = default; optional(optional const& rhs) requires /* copyable */ : engaged(rhs.engaged) { if (rhs.engaged) { new (&value) T(rhs.value); } } } </pre>

<pre> ~base_payload() { if (engaged) value.~T(); } }; template <typename T> struct base_payload<T, true> { struct empty { }; union { empty _; T value; }; bool engaged = false; constexpr base_payload() = default; constexpr base_payload(base_payload const&) = default; ~base_payload() = default; }; template <bool allow_copying> struct copy_enabler { copy_enabler() = default; copy_enabler(copy_enabler const&) = delete; ~copy_enabler() = default; }; template <> struct copy_enabler<true> { copy_enabler() = default; copy_enabler(copy_enabler const&) = default; ~copy_enabler() = default; }; template <typename T> class optional : private copy_enabler</* copyable */> , private base_payload<T> { public: using base_payload<T>::base_payload; optional(optional const&) = default; ~optional() = default; }; </pre>	<pre> } ~optional() requires /* trivially destructible */ = default; ~optional() { if (engaged) value.~T(); } }; </pre>
---	--

The code on the right isn't just much shorter, it also expresses the programmer's intent much more clearly, it's easier to read, it's easier to write. As a result of being much more condensed, it's easier to absorb. Nothing needs to be duplicated. It's strictly superior in all respects: all we have to do is *let* this implementation allow `optional<int>` to be considered trivially copyable and trivially destructible.

With this proposal, Casey Carter presents a P0602-compliant implementation of *all* of `std::optional`'s special member functions in just **93** lines of code [1]. His equivalent implementation in MSVC's standard library is ~350 lines. The difference in length surely undersells the difference in comprehensibility.

3. Design Considerations

This proposal seems to advocate for allowing a type to have multiple destructors. It is important to note that we already have the ability to overload special member functions, including destructors, in the working paper. In all cases, only one, single, *usable* special member will lead to code generation - all less constrained alternatives will be discarded. Each class template specialization will still have only a single destructor. Casey's full implementation linked here and the simplified example above are already legal code; the problem with both is that they are simply never trivially copyable. What this proposal aims for is to simply adjust the higher-level rules surrounding triviality to accommodate this new ability that Concepts gave us.

An alternative design would involve creating new syntax to allow for the creation of a conditionally-trivial destructor. A few such syntaxes were suggested on the mailing list [2] [3], the purported advantage of these being that we maintain the invariant of having just one declared destructor within the class. However, whatever new syntax is preferred, it would be end up being unique to destructors. There is no concern about having multiple copy constructors - this was already possible since C++11. Hence, a special syntax, for a special case, that needs to be specially learned.

A different design might involve restricting class types to a single user-declared destructor, but adjust the rules for implicit destructor generation to generate one anyway if the user-declared destructor were constrained. This has its own problems as well. The rules for implicit special member generation are already complex. To properly apply to the motivating example, the user-declared destructor for `std::optional` would have to be constrained on **not** trivially destructible types. Such a rule would negatively impact readability and comprehensibility, as negated concepts are inherently odd and disallowing defaulted functions would be a special restriction unique to destructors.

The advantage of this proposal when set against any alternative is that a constrained destructor behaves identically to a constrained copy constructor which behaves identically to any other constrained member or non-member function. There are no special cases. The code simply behaves the way that one would expect. This situation is not so special as to merit the creation of a special exception.

4. References

- [1] <https://godbolt.org/g/gg1kQX>
- [2] <http://lists.isocpp.org/ext/2017/03/2467.php>
- [3] <http://lists.isocpp.org/ext/2017/03/2365.php>